Proprietary

UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET

STEGANOGRAFIJA U QR KODOVIMA

- SEMINARSKI RAD -

Studijski program: Računarstvo i informatika
Modul: Bezbednost računarskih sistema
Predmet: Digitalna forenzika

Student:

Andrea Valčić, br. Ind. 1928

Profesori:

prof. dr Dejan Rančić
prof. dr Bratislav Predić

Niš, April 2026. godine

SADRŽAJ

1. UVOD ... 3

2. KONCEPT STEGANOGRAFIJE ... 3

2.1. ŠTA JE STEGANOGRAFIJE I KADA SE PRVI PUT POJAVILA? ... 3
2.2. VRSTE STEGANOGRAFIJE ... 3
2.3. RAZLIKA IZMEĐU STEGANOGRAFIJE I KRIPTOGRAFIJE ... 4
2.4. PRIMENA U DIGITALNOJ FORENZICI ... 4

3. QR KODOVI ... 4

3.1. STRUKTURA I FORMAT QR KODA ... 4
3.2. NIVOI KOREKCIJE GREŠAKA ... 4
3.3. POGODNOST QR KODOVA ZA STEGANOGRAFIJU ... 5

4. LSB STEGANOGRAFIJA ... 5

4.1. PRINCIP RADA LSB SUBSTITUCIJE ... 5
4.2. KAPACITET SKRIVANJA PODATAKA ... 6
4.3. STEGANALIZA I HI-KVADRAT TEST ... 6
4.4. OGRANIČENJA I SLABOSTI METODE ... 7

5. RAZVOJNO OKRUŽENJE, PROGRAMSKI JEZIK I TEHNOLOGIJE KORIŠĆENE ZA IZRADU PRAKTIČNOG DELA ... 7

5.1. VISUAL STUDIO CODE ... 7
5.2. PYTHON ... 7
5.2.1 Biblioteke u Python-u ... *7*
5.3. REED-SOLOMON KOREKCIJA GREŠAKA ... 8
5.4. SHA-256 ... 9

6. IMPLEMENTACIJA PRAKTIČNOG DELA ... 10

6.1. PAYLOAD STRUKTURA ... 10
6.2. GENERISANJE QR KODA I UGRAĐIVANJE PORUKE ... 10
6.3. PROCENA KAPACITETA ... 11
6.4. UGRAĐIVANJE PORUKE ... 11
6.5. EKSTRAKCIJA I VERIFIKACIJA PORUKE ... 12
6.6. FORENZIČKA ANALIZA ... 13

7. REZULTATI I ANALIZA ... 14

7.1. PRIKAZ GENERISANIH QR KODOVA ... 14
7.2. FORENZIČKA ANALIZA LSB RAVNI ... 15
7.3. ANALIZA REZULTATA ... 16

8. ZAKLJUČAK ... 18

LITERATURA ... 19

1. UVOD

U doba digitalne komunikacije, zaštita podataka postala je jedan od ključnih izazova informacione bezbednosti. Steganografija, kao disciplina koja se bavi skrivanjem informacija unutar digitalnih medija, predstavlja moćan alat koji se koristi kako u legitimne svrhe zaštite privatnosti, tako i u kontekstu digitalnih forenzičkih istraga. Motivacija za ovaj rad proizilazi iz potrebe da se razumeju i istraže tehnike prikrivanja podataka koje se sve češće pojavljuju u savremenim forenzičkim istragama, a čije poznavanje predstavlja neophodnu kompetenciju svakog stručnjaka u oblasti digitalne bezbednosti. Cilj seminarskog rada je analiza teorijskih osnova steganografije i praktična demonstracija kako se tajna poruka može sakriti unutar QR koda na način koji je vizuelno neprimetljiv i funkcionalno neoštećen. Praktični deo obuhvata implementaciju alata u programskom jeziku Python, koji omogućava generisanje QR kodova, ugrađivanje skrivenih poruka putem LSB steganografije, ekstrakciju podataka, kao i osnovnu forenzičku analizu pomoću hi-kvadrat testa.

2. Koncept steganografije

2.1. Šta je steganografije i kada se prvi put pojavila?

Steganografija potiče od grčkih reči *steganos*, što znači skriven i *graphia*, što označava pisanje, a kao disciplina postoji još od antičkog doba. Kroz istoriju, tehnike skrivanja poruka kretale su se od pisanja nevidljivim mastilom, pa sve do sofisticiranih digitalnih metoda koje se danas koriste. U modernom kontekstu, steganografija označava skup tehnika uz pomoć kojih se tajna informacija ugrađuje u digitalni medij kao što je slika, audio ili video zapis, na načinn da njeno prisustvo ostane neprimetno.

2.2. Vrste steganografije

U steganografiji, originalni medij u koji se ugrađuje skriveni sadržaj naziva se *cover* medij, dok se rezultujući medij sa ugrađenom porukom naziva *stego* medij. U zavisnosti od vrste nosača koji se koristi, steganografija se deli na nekoliko osnovnih kategorija. Vizuelna steganografija podrazumeva skrivanje podataka unutar digitalnih slika, što je ujedno i najrasprostranjeniji oblik. Audio steganografija koristi zvučne zapise kao nosače.Video steganografija proširuje iste principe na sekvence slika. Tekstualna steganografija manipuliše formatiranjem ili strukturom teksta, dok steganografija u protokolima skriva podatke unutar mrežnog saobraćaja.

2.3. Razlika između steganografije i kriptografije

Iako obe discipline imaju za cilj zaštitu informacija, njihovi pristupi se suštinski razlikuju. Kriptografija podatke čini nečitljivim. Kod nje, poruka je vidljiva ali sadržaj nije moguće razumeti bez ključa. Steganografija, nasuprot tome, skriva samo postojanje poruke. U ovom slučaju, sadržaj nije šifrovan ali primalac koji ne zna gde da traži sakrivenu poruku neće ni znati da ona postoji. U praksi se ove dve tehnike često kombinuju radi veće sigurnosti.

2.4. Primena u digitalnoj forenzici

U praksi se steganografija pojavljuje u dva konteksta. Prvi je zloupotreba pri kojem je napadači koriste za prenos ukradenih podataka iz sistema, za komuniciraju bez znanja bezbednosnih alata ili za sakrivanje dokaza o svojim aktivnostima. Drugi kontekst je istraga gde forenzičari koji analiziraju digitalne dokaze moraju da budu upoznati sa činjenicom da ovakve tehnike postoje i da budu u stanju da ih otkriju. Zbog toga je razumevanje steganografije obavezni deo znanja svakog stručnjaka u oblasti digitalne bezbednosti.

3. QR kodovi

3.1. Struktura i format QR koda

QR kod (*Quick Response*) je dvodimenzionalni barkod koji podatke koduje u matrici crnih i belih modula. Svaki QR kod sadrži nekoliko fiksnih strukturnih elemenata:

- Finder patterns - kvadrati u tri ugla koji skeneru omogućavaju prepoznavanje pozicije i orijentacije koda
- Timing patterns - naizmenični redovi crnih i belih modula koji definišu mrežu koordinata
- Alignment patterns - pomažu pri ispravljanju iskrivljene slike
- Oblast za podatke - preostali moduli koji sadrže kodirani sadržaj i podatke za korekciju grešaka

3.2. Nivoi korekcije grešaka

Kada se govori o QR standardu, mogu se definisati četiri nivoa korekcije grešaka. Svaki od njih garantuje oporavak određenog procenta oštećenih podataka:

- L - do 7% oštećenih podataka
- M - do 15% oštećenih podataka
- Q - do 25% oštećenih podataka
- H - do 30% oštećenih podataka

Što je nivo viši, to je kod otporniji na oštećenja. Kod ostaje čitljiv čak i kada je dobar deo površine uništen ili prekriven. Ova otpornost se postiže Reed-Solomon algoritmom koji uz stvarne podatke upisuje i dodatne informacije za oporavak. Kao posledica, kodovi sa višim nivoom korekcije imaju veću matricu i više modula.

3.3. Pogodnost QR kodova za steganografiju

QR kodovi su pogodni nosači skrivenih informacija iz nekoliko razloga. Nivo korekcije H dozvoljava da do 30% modula bude izmenjeno, a kod i dalje ostaje čitljiv. To znači da se mogu napraviti veoma male, gotovo neprimetne izmene na pikselima slike, konkretno menjanjem poslednjeg bita vrednosti svakog piksela, a da QR kod i dalje funkcioniše normalno. Ovakve izmene su vizuelno nevidljive jer pomeraju vrednost piksela za svega jedan nivo intenziteta. Osim toga, QR kodovi se sastoje samo od crnih i belih površina, što takve minimalne promene čini još teže uočljivim. Pošto se QR kodovi sreću svuda oko nas (na plakatima, proizvodima, dokumentima) niko ih ne doživljava kao sumnjive, što ih čini pogodnim medijem za prikrivanje informacija.

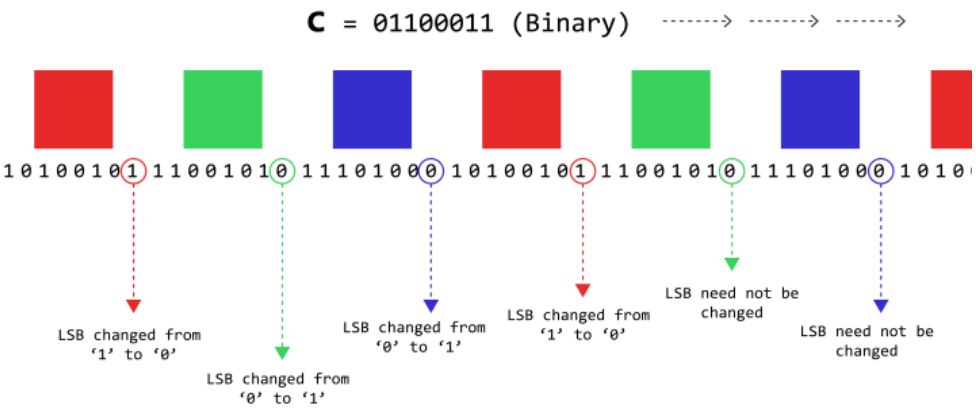
4. LSB steganografija

LSB (*Least Significant Bit*) steganografija je tehnika skrivanja podataka koja se zasniva na modifikaciji poslednjeg bita vrednosti svakog piksela digitalne slike. Predstavlja jedan od najrasprostranjenijih oblika digitalne steganografije zbog svoje jednostavnosti, efikasnosti i činjenice da izmene koje unosi u sliku ostaju vizuelno neprimetne.

Za razliku od naprednijih tehnika koje rade sa matematičkim transformacijama slike, LSB steganografija direktno menja vrednosti piksela, što je čini jednostavnom za implementaciju ali i lakše uočljivom pri detaljnoj analizi. Njena glavna prednost je visok kapacitet skrivanja zato što svaki piksel može da nosi jedan bit informacije. Dok je osnovna slabost osetljivost na bilo kakvu naknadnu obradu slike, poput kompresije ili skaliranja, koja može da uništi skrivenu poruku. Zbog toga se LSB steganografija koristi isključivo sa nekomprimovanim formatima, od kojih je PNG najčešći izbor u praksi.

4.1. Princip rada LSB substitucije

Svaki piksel digitalne slike predstavlja se brojem od 0 do 255, gde 0 označava crnu, a 255 belu boju. U binarnom zapisu taj broj ima 8 bitova, od kojih poslednji, najmanji bit, poznat kao LSB (*Least Significant Bit*), ima najmanji uticaj na vizuelni izgled piksela. Promena tog jednog bita pomera vrednost piksela za svega jedan nivo intenziteta, što je ljudskom oku potpuno nevidljivo. LSB steganografija koristi upravo tu osobinu - u poslednji bit svakog piksela upisuje se jedan bit skrivene poruke, piksel po piksel, dok se cela poruka ne ugradi u sliku. U nastavku, na slici 1 prikazan je primer binarnog zapisa piksela kojim se demonstrira stanje pre i posle LSB izmene.



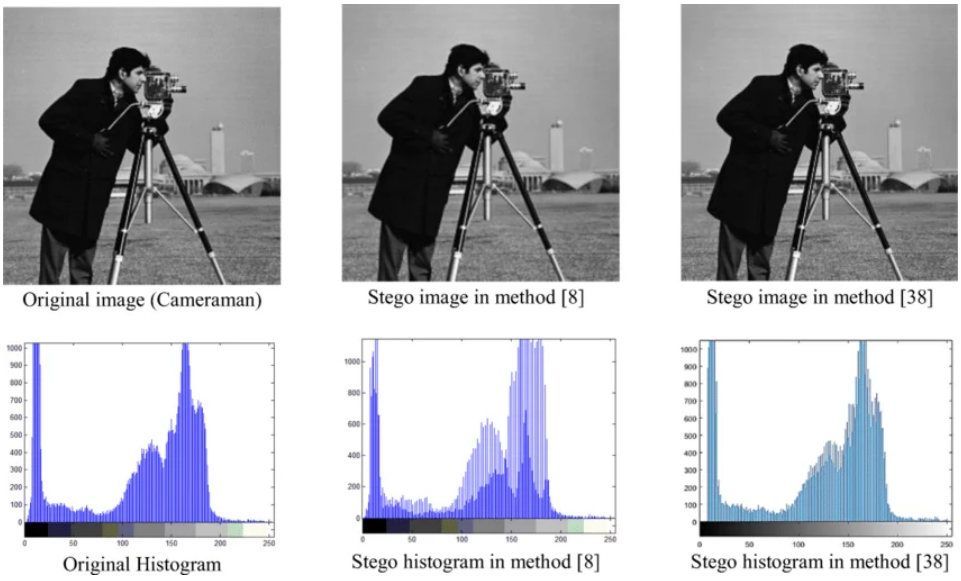
Slika 1. Prikaz binarnog zapisa piksela pre i posle LSB izmene

4.2. Kapacitet skrivanja podataka

Kapacitet nosača direktno zavisi od veličine slike. Pošto svaki piksel može da sakrije jedan bit, a jedan bajt se sastoji od 8 bitova, slika od N piksela može da primi N/8 bajtova skrivenog sadržaja. Deo tog prostora troši se na zaglavlje payload-a , poznat i pod nazivom magični marker, dužinu poruke, broj verzije i SHA-256 heš, što ukupno iznosi 41 bajt fiksnog overhead-a. Ostatak je slobodan za stvarnu poruku.

4.3. Steganaliza i hi-kvadrat test

Steganaliza je disciplina suprotna steganografiji. Njen cilj je otkrivanje skrivenog sadržaja. Jedan od osnovnih alata je hi-kvadrat test, koji proverava da li su LSB vrednosti piksela raspoređene na prirodan ili neprirodan način. U običnoj slici, raspodela poslednjih bitova prati određeni statistički obrazac. Nakon LSB ugrađivanja, taj obrazac se menja jer su bitovi zamenjeni podacima poruke, što hi-kvadrat test može da detektuje kao anomaliju. Na slici 2 prikazan je histogram LSB vrednosti za originalnu sliku kao i njene stego slike.



Slika 2. Histogram LSB vrednosti za originalnu i stego slike

4.4. Ograničenja i slabosti

LSB steganografija je jednostavna i efikasna ali ima nekoliko slabosti. Osetljiva je na bilo kakvu obradu slike. Kompresija, skaliranje ili korekcija boja mogu uništiti skrivenu poruku jer menjaju vrednosti piksela. Pored toga, statistički alati za steganalizu mogu da detektuju prisustvo skrivenog sadržaja čak i bez poznavanja ključa. Metoda je stoga najpouzdanija kada se koristi sa nekomprimovanim formatima poput PNG i kada slika nije naknadno obrađivana.

5. Razvojno okruženje, programski jezik i tehnologije korišćene za izradu praktičnog dela

5.1. Visual Studio Code

Tokom razvoja sistema korišćeno je programsko okruženje Visual Studio Code, koje pruža podršku za veliki broj programskih jezika. Ovo okruženje se široko primenjuje kako u razvoju veb aplikacija, tako i u izradi aplikacija za mobilne uređaje i tablete. Visual Studio Code dolazi sa ugrađenom podrškom za JavaScript, TypeScript i Node.js, dok se dodatna funkcionalnost obezbeđuje kroz bogat ekosistem ekstenzija za druge programske jezike i izvršna okruženja, kao što su C++, C#, Java, Python, PHP, Go i .NET.

5.2. Python

Prilikom izrade … korišćen je Python programski jezik. Ovaj programski jezik je razvio Gvido van Rosum početkom devedeseth godina i predstavlja programski jezik visokog nivoa opšte namene. Podržava imperativni, objektno-orjentisan i funkcionalni stil programiranja. Python se odlikuje preglednom i jednostavnom sintaksom, što ga čini pogodnim za brzo učenje i razvoj aplikacija. Programi napisani u ovom jeziku izvršavaju se pomoću interpretatora, uz koji dolazi i obimna standardna biblioteka modula.

5.2.1 Biblioteke u Python-u

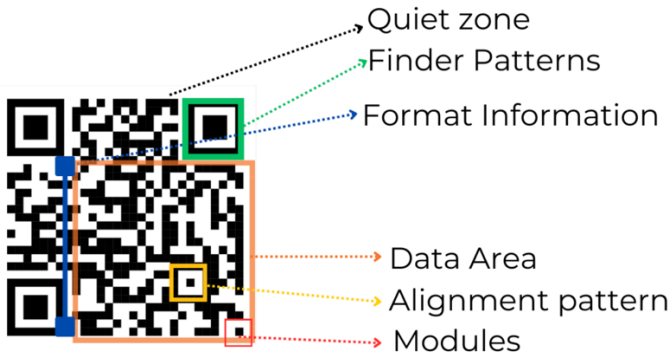
Python raspolaže bogatim ekosistemom modula i paketa koji nude širok spektar funkcionalnosti i omogućavaju programerima da realizuju različite zadatke bez potrebe za pisanjem koda od nule. Ove biblioteke sadrže unapred definisane klase, funkcije i rutine koje se koriste za razvoj aplikacija, automatizaciju procesa, obradu i analizu podataka, matematičke proračune i druge složene operacije. Ekosistem Python biblioteka obuhvata brojne oblasti, uključujući razvoj veb aplikacija, analizu podataka, mašinsko učenje, obradu slika i naučna računanja, što značajno doprinosi popularnosti ovog programskog jezika i olakšava efikasnu implementaciju kompleksnih rešenja.

Python biblioteke korišćene u praktičnom delu rada:

- **qrcode**: omogućava generisanje QR kodova sa podrškom za izbor nivoa korekcije grešaka
- **Pillow**: koristi se za obradu i manipulaciju slikama (čitanje, pisanje i modifikacija vrednosti piksela)
- **NumPy**: pruža efikasne operacije nad nizovima piksela pri ugrađivanju i analizi skrivenog sadržaja
- **hashlib**: generisanje SHA-256 heša za verifikaciju integriteta skrivene poruke
- **os**: omogućavaupravljanje putanjama fajlova pri čuvanju generisanih slika

5.3. Reed-Solomon korekcija grešaka

Reed-Solomon je algoritam za detekciju i ispravljanje grešaka koji se široko primenjuje u digitalnim komunikacijama i sistemima za skladištenje podataka. U QR standardu, Reed-Solomon se koristi za generisanje redundantnih podataka koji se dodaju uz originalni sadržaj koda. Ti redundantni podaci omogućavaju rekonstrukciju oštećenih ili nedostajućih informacija, sve do određenog praga koji zavisi od izabranog nivoa korekcije grešaka.



Slika 3. QR kod sa označenom oblašću podataka

Algoritam funkcioniše na principu polinomske aritmetike nad Galoisovim poljima. Originalni podaci se tretiraju kao koeficijenti polinoma, a redundantni blokovi se računaju tako da omogućavaju jednoznačnu rekonstrukciju originalnog polinoma čak i kada deo koeficijenata nedostaje. U praksi to znači da QR kod sa nivoom korekcije H može da izgubi ili ima oštećeno do 30% svojih modula, a i dalje će biti ispravno dekodovan.

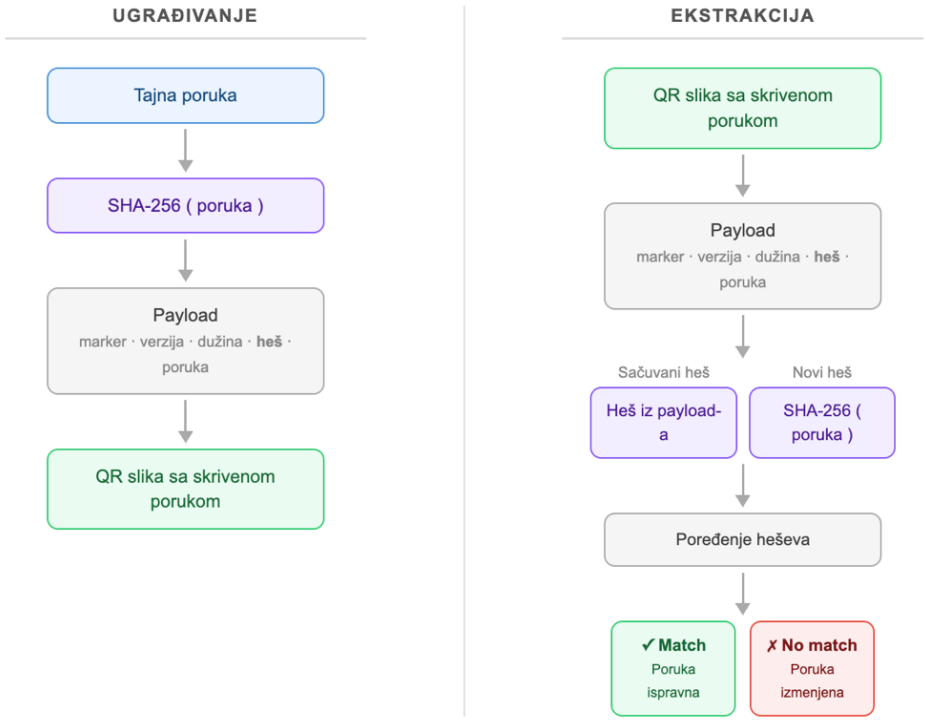
U kontekstu ovog rada, Reed-Solomon redundantnost je ključna pretpostavka na kojoj počiva cela implementacija. LSB izmene piksela su sa stanovišta QR čitača ekvivalentne fizičkom oštećenju modula i upravo tolerancija Reed-Solomon algoritma osigurava da stego QR kod ostane funkcionalan nakon ugrađivanja skrivene poruke.

5.4. SHA-256

SHA-256 (*Secure Hash Algorithm 256*) je kriptografska heš funkcija iz SHA-2 familije. Razvila ju je Nacionalna agencija za bezbednost SAD (*NSA*). Za bilo koji ulaz proizvoljne dužine generiše jedinstveni otisak fiksne dužine od 256 bitova, poznat kao heš ili digest. SHA-256 poseduje nekoliko ključnih osobina koje ga čine pogodnim za primenu u ovom radu.

Jednosmernost znači da je iz heša praktično nemoguće rekonstruisati originalni ulaz. Otpornost na kolizije garantuje da je praktično nemoguće pronaći dva različita ulaza koji daju isti heš. Avalanche efekat označava osobinu da i najmanja promena ulaza kao što je izmena jednog slova u poruci, rezultuje potpuno drugačijim hešom.

U praktičnom delu rada, SHA-256 je korišćen za proveru integriteta ekstrahovane poruke. Pre ugrađivanja, heš tajne poruke se računa i upisuje u payload zajedno sa samom porukom. Nakon ekstrakcije, heš se ponovo računa i upoređuje sa sačuvanim. Poklapanje vrednosti garantuje da poruka nije izmenjena niti oštećena tokom prenosa. U nastavku, na slici 4 prikazan je SHA-256 tok koji se verifikuje integritet poruke.



Slika 4. SHA-256 tok verifikacije integriteta poruke

6. Implementacija praktičnog dela

Praktični deo rada obuhvata implementaciju alata u programskom jeziku Python koji omogućava generisanje QR kodova sa ugrađenom skrivenom porukom, ekstrakciju te poruke, kao i osnovnu forenzičku analizu rezultujuće slike. Cela implementacija nalazi se u skripti *qr_stego.py* i organizovana je kroz šest funkcionalnih celina.

6.1. Payload struktura

Pre ugrađivanja, tajna poruka se pakuje u strukturirani payload koji sadrži sve informacije potrebne za ekstrakciju i verifikaciju. Payload se formira na sledeći način:

Magični marker **0xDEADBEEF** upisuje se na početak svakog payload-a i služi kao potpis. Prilikom ekstrakcije njegova odsutnost odmah signalizira da slika ne sadrži skrivenu poruku. Iza markera sledi broj verzije, dužina poruke kao 4-bajtni ceo broj, SHA-256 heš za proveru integriteta i na kraju sama poruka u UTF-8 kodiranju. Ukupan overhead zaglavlja iznosi 41 bajt.

```
    Payload structure (bytes):
      MARKER (4) | VERSION (1) | MSG_LEN (4, big-endian) | SHA256(secret) (32) | MSG
    """
    secret_bytes = secret.encode("utf-8")
    sha = hashlib.sha256(secret_bytes).digest()

    payload = (
        MARKER
        + bytes([VERSION])
        + len(secret_bytes).to_bytes(4, "big")
        + sha
        + secret_bytes
    )
```

Slika 5. Definicija payload-a

6.2. Generisanje QR koda i ugrađivanje poruke

QR kod se generiše korišćenjem qrcode biblioteke sa nivoom korekcije grešaka H, koji obezbeđuje najveću redundantnost i time najveći kapacitet za skrivanje podataka:

```
def generate_qr(payload: str, error_level=qrcode.constants.ERROR_CORRECT_H, box_size=10, border=4) -> Image.Image:
    """Generate a QR with the given error correction level."""
    qr = qrcode.QRCode(
        version=None,
        error_correction=error_level,
        box_size=box_size,
        border=border,
    )
    qr.add_data(payload)
    qr.make(fit=True)
    img = qr.make_image(fill_color="black", back_color="white").convert("L")
    return img
```

Slika 6. Definicija metode za generisanje QR koda

Parametar **version=None** znači da biblioteka automatski bira minimalnu verziju QR koda dovoljnu za zadati sadržaj. Konverzija u mod **"L"** prevodi sliku u nijanse sive, što je neophodan format za LSB manipulaciju vrednostima piksela.

6.3. Procena kapaciteta

Pre ugrađivanja poruke vrši se procena raspoloživog kapaciteta kako bi se osiguralo da slika može da primi ceo payload. Kapacitet se računa na osnovu ukupnog broja piksela slike:

```python

def estimate_capacity(img: Image.Image) -> int:
    """Estimate how many secret bytes can be hidden (conservative: 10% of modules)."""
    bits = _img_to_bits(img)

    # Count dark modules (1-bits are white in PIL "1" mode)
    total_modules = len(bits)

    # LSB of every 8th pixel cluster is used
    usable_bits = total_modules // 8

    overhead = len(MARKER) + 1 + 4 + 32  # marker(4) + version(1) + msg_length(4) + sha256(32)

    return max(0, usable_bits - overhead)

```

Slika 7. Definicija metode za računanje kapaciteta

Pošto svaki piksel može da sakrije jedan bit, a jedan bajt se sastoji od 8 bitova, ukupan kapacitet u bajtovima dobija se deljenjem broja piksela sa 8. Od tog broja oduzima se 41 bajt fiksnog overhead-a zaglavlja, čime se dobija stvarni prostor raspoloživ za tajnu poruku.

6.4. Ugrađivanje poruke

Ugrađivanje se vrši LSB substitucijom. Poslednji bit vrednosti svakog piksela zamenjuje se jednim bitom payload-a. Centralni deo funkcije **embed_message** je sledeća petlja:

```python

    # Modify LSB of each pixel
    for i, bit in enumerate(payload_bits):
        flat[i] = (flat[i] & 0xFE) | bit

```

Slika 8. Implementacija LSB substitucijom kroz petlju

Operacija **& 0xFE** postavlja poslednji bit piksela na 0, a **| bit** upisuje željenu vrednost. Na ovaj način vrednost piksela se menja za najviše ±1, što je vizuelno neprimetno. Pre same petlje vrši se provera da li slika ima dovoljno piksela za ceo payload:

```python

    if len(payload_bits) > len(flat):
        raise ValueError(
            f"Secret too long! Need {len(payload_bits)} bits, image has {len(flat)} pixels."
        )

```

Slika 9. Prikaz provera veličine payload-a

6.5. Ekstrakcija i verifikacija poruke

Ekstrakcija poruke se vrši čitanjem LSB vrednosti svakog piksela i rekonstrukcijom bajtova iz dobijenog niza bitova:

```python

    # Pull LSBs
    lsbs = (flat & 1).astype(np.uint8)
    raw = _bits_to_bytes(lsbs)

```

Slika 10. Prikaz čitanja vrednosti svakog piksela i rekonstrukcija u bajtove

Nakon rekonstrukcije payload-a vrše se tri provere pre nego što se poruka prihvati kao validna. Prvo se proverava prisustvo magičnog markera:

```python

    # Verify marker
    if raw[:4] != MARKER:
        raise ValueError("No steganographic payload found (marker mismatch).")

```

Slika 11. Prikaz verifikacije markera

Zatim se proverava broj verzije, čita dužina poruke i na kraju se vrši SHA-256 verifikacija integriteta:

```python

    # Integrity check
    actual_sha = hashlib.sha256(msg_bytes).digest()
    if actual_sha != stored_sha:
        raise ValueError("Integrity check failed - message may be corrupted or tampered.")

```

Slika 12. Prikaz verifikacije integriteta

Ako sve tri provere prođu, vrši se dekodiranje poruke iz UTF-8 i vraća se rezultat.

6.6. Forenzička analiza

Metoda **analyze_image** vrši osnovnu steganalizu slike računanjem hi-kvadrat statistike nad LSB vrednostima piksela. Prvobitno se broje pikseli čiji je LSB jednak 0 i oni čiji je LSB jednak 1, nakon čega se izračuna statistika:

```
    chi2 = ((ones - expected)**2 + (zeros - expected)**2) / expected

    return {
        "pixels": len(flat),
        "lsb_ones": ones,
        "lsb_zeros": zeros,
        "lsb_ratio": round(ones / len(lsbs), 4),
        "chi2_statistic": round(chi2, 4),
        "anomaly_detected": chi2 > 3.84,  # p=0.05 threshold
        "capacity_bytes": len(flat) // 8,
    }

```

Slika 13. Prikaz računanja hi-kvadrta statistike

U slučaju da slika ne sadrži skriveni sadržaj, očekuje se približno jednaka raspodela nula i jedinica u LSB ravni. Nakon LSB ugrađivanja, ta raspodela se menja jer su bitovi zamenjeni podacima poruke. Vrednost praga 3.84 odgovara nivou značajnosti p=0.05. U slučaju da je hi-kvadrat statistika veća od ove vrednosti, raspodela LSB bitova statistički značajno odstupa od očekivane i može ukazivati na prisustvo skrivenog sadržaja.

7. Rezultati i analiza

Nakon pokretanja skripte *qr_stego.py*, u direktorijumu projekta generišu se tri slike koje zajedno ilustruju ceo proces steganografskog ugrađivanja i na taj način pružaju vizuelnu i statističku analizu rezultata.

7.1. Prikaz generisanih QR kodova

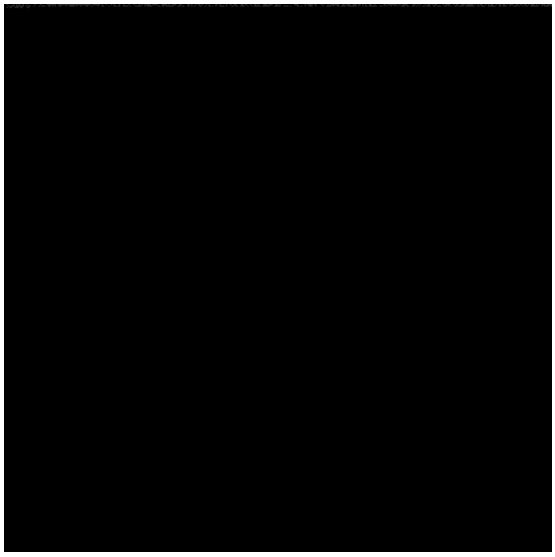
Na slici 13. prikazan je čist QR kod koji koduje javnu poruku bez ikakvog skrivenog sadržaja.

Slika 13. QR kod bez skrivenog sadržaja

Druga slika, slika 14, je vizuelno identična prvoj, ali sadrži tajnu poruku koja je ugrađena.

Slika 14. QR kod sa ugrađenom tajnom porukom

Na samom kraju, treća i poslednja slika prikazuje razliku između prve dve slike pojačanu 40 puta kako bi se izmenjeni pikseli mogli uočiti.



Slika 14. Prikaz razliike između prve dve slike

7.2. Forenzička analiza LSB ravni

LSB ravan predstavlja sliku koja se dobija izdvajanjem poslednjeg bita vrednosti svakog piksela. Vizualizacijom te ravni dobija se crno-bela slika u kojoj bele tačke označavaju piksele čiji je LSB jednak 1, a crne tačke piksele čiji je LSB jednak 0. U slučaju obične fotografije, raspodela bi izgledala kao šum bez vidljivog obrasca. Kod stego slike, u oblasti gde je poruka ugrađena pojavljuje se primetno drugačiji obrazac koji odražava strukturu payload-a.

Slika 14. zumiranjem pri samom vrhu slike vizualno se potvrđuje gde su izmene nastale. Svaka bela tačka označava piksel čija je vrednost promenjena tokom ugrađivanja. Pojačavanje razlike 40 puta neophodno je jer su stvarne promene vrednosti piksela jednake svega ±1, što je bez pojačavanja potpuno nevidljivo.

Hi-kvadrat test proverava da li je raspodela LSB vrednosti statistički uniformna. Test poredi stvarni broj nula i jedinica u LSB ravni sa očekivanim brojem pri uniformnoj raspodeli:

```
chi2 = ((ones - expected)**2 + (zeros - expected)**2) / expected
anomaly_detected: chi2 > 3.84
```

Vrednost praga 3.84 odgovara nivou značajnosti p=0.05. Važno ograničenje ovog testa pri analizi QR kodova je što oni zbog svoje binarne strukture uvek pokazuju visoku hi-kvadrat vrednost; i originalni i stego kod. To znači da hi-kvadrat test nije pouzdan indikator prisustva skrivenog sadržaja kada je nosač QR kod, za razliku od fotografija gde daje pouzdanije rezultate.

7.3. Analiza rezultata

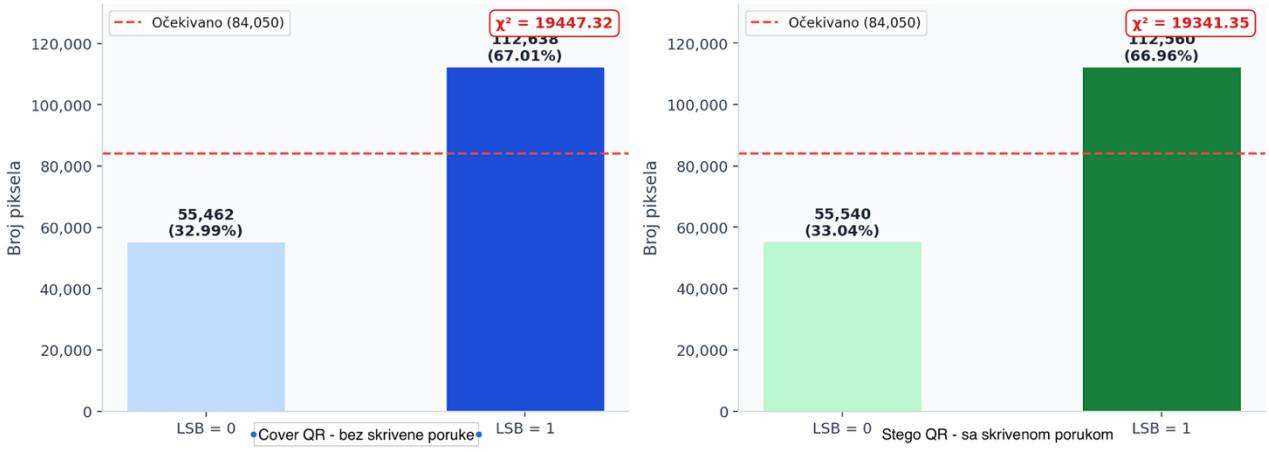
Skripta je testirana sa javnom porukom *https://www.elfak.ni.ac.rs/* i tajnom porukom dužine 64 bajta. Generisani QR kod dimenzija je 450×450 piksela, što daje ukupan kapacitet od 25.268 bajtova slobodnih za skrivanje podataka nakon oduzimanja overhead-a zaglavlja. Na slici 15 predstavljen je tabela koja sadrži statističke parametre LSB steganografije na QR kodu.

Parametar	Vrednost
Dimenzije slike	450 × 450 px
Ukupan broj piksela	202.500
Kapacitet (bez overhead-a)	25.268 bajtova
Veličina tajne poruke	64 bajta
Iskorišćenost kapaciteta	0,25%
Broj izmenjenih piksela	463
Procenat izmenjenih piksela	0,23%

Slika 15. Tabela sa statističkim parametrima LSB steganografije na QR kodu

Broj izmenjenih piksela direktno odgovara broju bitova u payload-u. Svaki bit zahteva tačno jedan piksel. Od ukupno 202.500 piksela, izmenjeno je svega 463, što potvrđuje da LSB steganografija unosi minimalne i vizuelno neprimetne izmene u sliku.

Na slici 16 prikazan je histogram LSB vrednosti za oba QR koda. Histogram pokazuje raspodelu piksela čiji je poslednji bit jednak 0 i onih čiji je jednak 1, uz crvenu isprekidanu liniju koja označava očekivanu uniformnu raspodelu. Kod oba koda može se primetiti veći broj piksela sa LSB=1 u odnosu na LSB=0, što je posledica binarne prirode QR strukture, a ne prisustva skrivenog sadržaja. Razlika u raspodeli između originalnog i stego koda je minimalna, što jasno potvrđuje da ugrađivanje poruke nije statistički uočljivo kada je nosač QR kod. Iz istog razloga je kod oba koda prisutna visoka hi-kvadrat vrednost, što ukazuje na ograničenje hi-kvadrat testa pri analizi QR kodova u poređenju sa fotografijama gde daje pouzdanije rezultate.



Slika 16. Histogram LSB vrednostiza cover i stego QR kod.

Ekstrakcija same poruke je u potpunosti izvršena uspešno. SHA-256 verifikacija potvrđuje integritet, dok dekodovana poruka u potpunosti odgovara originalnoj. Oba QR koda, originalni i stego, su ostali funkcionalni i mogu se uspešno skenirati standardnim QR čitačem, što potvrđuje da Reed-Solomon mehanizam korekcije grešaka apsorbuje unesene LSB izmene bez ikakvih posledica po čitljivost koda.

8. ZAKLJUČAK

U ovom seminarskom radu istražena je primena steganografije u kontekstu QR kodova, sa fokusom na LSB tehniku skrivanja podataka. Kroz teorijski deo rada analizirani su osnovni koncepti steganografije, struktura QR kodova i uloga Reed-Solomon algoritma, dok je praktični deo demonstrirao kako se ove tehnike mogu primeniti u realnom scenariju.

Ključni zaključak rada je da QR kodovi predstavljaju pogodan steganografski nosač upravo zbog ugrađenog mehanizma korekcije grešaka. Nivo korekcije H, koji toleriše do 30% oštećenih modula, stvara dovoljno prostora da LSB izmene ostanu nevidljive i za ljudsko oko i za QR čitač. Testiranje je potvrdilo da je moguće sakriti poruku uz izmenu manje od 0,23% piksela slike, pri čemu oba QR koda, originalni i stego, ostaju potpuno funkcionalni.

Sa stanovišta digitalne forenzike, rad pokazuje da detekcija steganografskog sadržaja u QR kodovima nije trivijalan zadatak. Hi-kvadrat test, koji se pokazuje kao pouzdan alat pri analizi fotografija, daje ograničene rezultate kada je nosač QR kod zbog njegove inherentno neuniformne strukture. Ovo ukazuje na potrebu za razvojem specijalizovanih metoda steganananalize prilagođenih binarnim slikama.

Implementirani alat može poslužiti kao osnova za dalja istraživanja, kao što su primena naprednijih steganografskih tehnika, kombinovanje steganografije sa kriptografijom radi veće sigurnosti, ili razvoj pouzdanijih forenzičkih alata za detekciju skrivenog sadržaja u QR kodovima.

LITERATURA

- https://en.wikipedia.org/wiki/Steganography
- https://en.wikipedia.org/wiki/QR_code
- https://en.wikipedia.org/wiki/Reed%E2%80%93Solomon_error_correction
- https://en.wikipedia.org/wiki/Least_significant_bit
- https://en.wikipedia.org/wiki/SHA-2
- https://en.wikipedia.org/wiki/Steganalysis
- https://en.wikipedia.org/wiki/Chi-squared_test
- https://pypi.org/project/qrcode/
- https://pypi.org/project/Pillow/
- https://numpy.org/doc/stable/
- https://docs.python.org/3/library/hashlib.html
- https://www.researchgate.net/publication/245096254_A_survey_of_steganographic_techniques
- https://www.thonky.com/qr-code-tutorial/introduction